RAG System Evaluation & Deployment Report

SECTIONS 7 – 10: RETRIEVAL, INTERFACE, LIMITATIONS, AND FUTURE FRAMEWORKS

7. Hybrid Search

Explanation of Hybrid Search

Hybrid search is an advanced information retrieval paradigm that combines separate retrieval mechanisms—typically keyword-based sparse retrieval and semantic-based dense retrieval—into a unified pipeline. While standard semantic search Excel at grasping conversational contexts, it often overlooks explicit key identifiers, serial codes, or specific jargon. Conversely, keyword matching captures exact strings perfectly but fails to understand conceptual synonyms. Hybrid search bridges this gap by querying both indices simultaneously and combining their respective outputs into a single, high-fidelity context vector for the Large Language Model (LLM).

Dense Retrieval Method

Dense retrieval maps text documents and user queries into a shared high-dimensional vector space using deep learning embedding models. Text sequences are mathematically structured into dense vectors where semantic relationships are preserved based on vector proximity. When a runtime query is received, the system evaluates the distance between the query vector and document vectors using mathematical metrics like Cosine Similarity or Dot Product.

Key Advantage: It excels at conceptual alignment and linguistic mapping, successfully matching queries like "tuition fees" with documents containing "financial layout" or "cost of enrollment" without requiring precise keyword overlap.

Sparse Retrieval Method

Sparse retrieval operates on lexical matching, traditionally utilizing the BM25 or TF-IDF algorithms. It treats documents as sparse frequency vectors over an entire vocabulary dictionary. The ranking is computed based on term frequency (TF) balanced against inverse document frequency (IDF), ensuring that highly specific search keywords contribute heavier weight than common words.

Key Advantage: It offers extreme precision for highly specialized nomenclature, proper nouns, unique identification hashes, or strict technical codes (e.g., specific course listings or administrative forms) that could otherwise be generalized or diluted within a dense multi-dimensional vector space.

Fusion Method

To blend the diverse ranking structures generated by dense and sparse systems, a linear combination using Alpha Weighting is employed. The raw dense similarity and sparse lexical relevance scores are normalized and combined using a controlled parameter coefficient (α).

$$Final\ Score = (\alpha \times Dense\ Score) + ((1 - \alpha) \times Sparse\ Score)$$

By configuring α (typically set between 0.5 and 0.7), the system optimizes for a semantic-heavy bias while keeping a robust exact-match fallback protection mechanism.

Comparison Between Dense-Only and Hybrid Search

Standard dense-only search frequently stumbles when dealing with specialized academic datasets or complex organizational directories. For instance, querying a precise catalog item or an alphanumeric course ID like "CS450" can result in a dense retriever returning generic computer science documents simply due to linguistic contextual similarities. Integrating sparse constraints prevents these errors, ensuring the exact match constraint forces the correct specific text chunks to rise to the top of the context window, ultimately suppressing LLM hallucinations.

Results for Test Queries

QUERY	DENSE-ONLY RESULT	HYBRID SEARCH RESULT	OBSERVED IMPROVEMENT
"Who teaches CS450?"	Returned a broad, unranked listing of global Computer Science faculty profiles.	Successfully isolated the direct syllabus document for CS450 along with the explicit assigned professor.	The sparse module cleanly captured the specific alpha-numeric identifier "CS450", while the dense mechanism localized the semantic context of "teaches".
"What are the graduation requirements?"	Returned the correct comprehensive institutional academic handbook pages.	Returned identical institutional academic handbook pages.	No substantial shift observed; dense semantic mapping was fully sufficient to decode the conceptual baseline of the query.
"Deadlines for the Fall NLP project"	Returned the generic administrative academic calendar and semester dates.	Pinpointed the exact course schedule outlining the "NLP assignment" and its explicit due date.	The sparse matching isolated "NLP", while the dense model properly unpacked the contextual, temporal constraint of "deadlines".

8. Streamlit GUI

User Interface and Components

The application frontend was designed and deployed utilizing the Streamlit framework, providing a clean, reactive, and highly responsive interactive web interface tailored for real-time natural language interaction.

[Screenshot 1: Main Conversational Chat Interface]

Visual interface capturing the user chat prompt, alternating message avatars, and the final streamed response.

The presentation layer consists of the following core architectural layout components:

- **Primary Chat Window:** A standard conversation layout utilizing native messaging blocks that smoothly render standard text, markdown tables, and dynamic code structures.
- **Sidebar Parameter Control:** A collapsible contextual pane allowing administrative toggles, including real-time adjustments of the α hybrid weights, retrieval limit controls (top-k), and pipeline health indicators.
- **Context Accordion Dropdown:** An expandable sub-interface attached directly beneath each response block that surfaces the exact document text snippets and source names retrieved from the vector database.

How the GUI Connects to the Pipeline

The Streamlit dashboard acts as a reactive orchestrator bound to the processing pipeline. When a user transmits an query string through the UI input interface, the workflow executes the following sequence:

- The text input fires a callback function, passing the string variables directly into the backend orchestration wrapper.
- The string is duplicated across the embedding encoder and the lexical token index to gather document chunks.
- The top scoring chunks are extracted, structured into an injection payload context window, and appended to the prompt template.
- The prompt payload is issued to the Gemini API, where the generated text stream is captured chunk-by-chunk and dynamically painted to the user interface.

Authentication and Chat History

[Screenshot 2: User Authentication & Access Screen]

Secure access barrier demanding validated profile credentials prior to unlocking the RAG workspace.

- **Authentication Controls:** To prevent unauthorized infrastructure consumption and protect the confidentiality of internal organizational documentation, a robust secure session wrapper forces user verification prior to rendering any dashboard elements.
- **Session Chat History:** Leveraging Streamlit's native state management framework (`st.session_state`), conversational histories are structurally preserved across active sessions. This allows sequential multi-turn dialogue, passing past conversation turns alongside new inputs to enable pronouns and complex follow-up questions to execute seamlessly.

9. Limitations

A rigorous assessment of the current production instance revealed several critical architectural and system limitations that affect operational performance:

- **Small Dataset Size:** The existing vector index operates on a restricted documentation footprint. This constraint prevents comprehensive edge-case answer generation and introduces clear boundaries to system domain knowledge.
- **Poor Website Structure:** The base unstructured web scrape contained volatile HTML layouts, causing minor noise ingestion where global site elements (footers, tracking nodes, and navigation tabs) occasionally merged into text chunks.
- **Weak Metadata Ingestion:** The harvested content contains minimal explicit structural metadata tagging. The absence of chronological dates, authors, or categorization scopes restricts the application's ability to apply advanced pre-retrieval filtering rules.
- **Retrieval Mistakes:** High linguistic overlap across disparate structural concepts can occasionally trigger false-positive retrievals, pushing actual high-relevance chunks out of the restricted LLM processing context window.
- **Gemini Model Boundaries:** The system remains fundamentally susceptible to minor downstream hallucinations if the prompt context lacks explicit grounding criteria. In these instances, the foundation model may supplement answers with pre-trained parameters.
- **Query Ambiguity:** Vague or non-specific user interactions (e.g., "Review the status of the standard policy") without contextual references often scatter retrieval focus, generating responses that lack specific alignment.

10. Future Work

To transition the existing framework toward an enterprise-scale architecture, future engineering iterations will prioritize the following tracks:

- **Enhanced Scraping & Coverage:** Migrate standard crawler configurations to fully robust, asynchronous web automation setups (such as Playwright or Scrapy pipelines) to extract clean text while ignoring navigation and layout boilerplate.
- **Automated Metadata Generation:** Incorporate high-throughput, low-latency micro-models during data ingestion to programmatically analyze incoming chunks and attach rich tags like document type, created dates, and topic domains.
- **Semantic Chunking Frameworks:** Replace arbitrary, fixed character-count splitting strategies with dynamic semantic boundary chunking. This parses text thresholds dynamically based on real shifts in thematic embedding continuity, ensuring complete logical thoughts remain unfragmented.
- **Empirical Hybrid Weight Optimization:** Establish automated testing routines to optimize the hybrid fusion α parameter by assessing retrieval accuracy against structured evaluation benchmarks.
- **Standardized RAGAS Evaluation Integration:** Deploy the RAGAS (Retrieval Augmented Generation Assessment) testing suite to track key performance indicators including Context Recall, Context Precision, Faithfulness, and Answer Relevance.
- **Production Containerization & Deployment:** Standardize the application stack using a Docker orchestration profile and deploy the system across scalable cloud runtimes (e.g., AWS ECS or Google Cloud Run) coupled with managed CI/CD pipelines.